

```
test@test-machine:~/ $ ./pmq_sender_ssmr
Sender MQ /mq-postman opened
Sender: sending message: [Hello there - 0]
Sender: sending message: [Hello there - 1]
Sender: sending message: [Hello there - 2]
Sender: sending message: [Hello there - 3]
Sender: sending message: [Hello there - 4]
Sender: sending message: [Hello there - 5]
Sender: sending message: [Hello there - 6]
Sender: sending message: [Hello there - 7]
Sender: sending message: [Hello there - 8]
Sender: sending message: [Hello there - 9]
Sender: Exit
test@test-machine:~/ $
```

Figure 3: A single sender process sending a series of messages to a message queue

```
test@test-machine:~/ $ ./pmq_receiver_ssmr
Receiver MQ /mq-postman opened
Receiver: blocked until message arrives...
Receiver: message received: [Hello there - 0]
Receiver: blocked until message arrives...
Receiver: message received: [Hello there - 2]
Receiver: blocked until message arrives...
Receiver: message received: [Hello there - 3]
Receiver: blocked until message arrives...
Receiver: message received: [Hello there - 5]
Receiver: blocked until message arrives...
Receiver: message received: [Hello there - 7]
Receiver: blocked until message arrives...
Receiver: message received: [Hello there - 9]
Receiver: blocked until message arrives...
^C
test@test-machine:~/ $
```

Figure 4: Receiver1 processes waiting and receiving messages from a message queue

In Figure 3, a single sender process sends a series of messages to the message queue. In Figure 4 and Figure 5, two receiver processes wait to receive messages from the same message queue. We can see that there is no specific order in which the receiver processes the messages sent by the sender.

## Setting priority to messages

One of the features of POSIX message queues is the ability to set priority levels for the messages when they are being sent. This helps the receiver process to receive the messages with the highest priority first among the other priorities, irrespective of the order in which they were sent by the sender process.

The code given below demonstrates the behaviour where a receiver processes the highest priority message in the message queue first, followed by the lower priority message in the queue. A sender process sends a series of messages with alternating priorities to the message queue.

```
test@test-machine:~/ $ ./pmq_receiver_ssmr2
Receiver MQ /mq-postman opened
Receiver: blocked until message arrives...
Receiver: message received: [Hello there - 1]
Receiver: blocked until message arrives...
Receiver: message received: [Hello there - 4]
Receiver: blocked until message arrives...
Receiver: message received: [Hello there - 6]
Receiver: blocked until message arrives...
Receiver: message received: [Hello there - 8]
Receiver: blocked until message arrives...
^C
test@test-machine:~/ $
```

Figure 5: Receiver2 processes waiting and receiving messages from a message queue

```
// pmq_sender_priority.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mqueue.h>

#define MQ_PERMISSIONS 0660
#define MQ_MAX_MESSAGES 10
#define MQ_MAX_MSG_SIZE 128
#define MQ_MSG_BUFFER_SIZE MQ_MAX_MSG_SIZE + 10

int main(int argc, char **argv)
{
    mqd_t mqd_sender;
    struct mq_attr attr;
    const char *mq_name = "/mq-postman";
    unsigned int msg_priority;
    char tx_buffer[MQ_MSG_BUFFER_SIZE] = {"Hello there
- "};

    attr.mq_flags = 0;
    attr.mq_maxmsg = MQ_MAX_MESSAGES;
    attr.mq_msgsize = MQ_MAX_MSG_SIZE;
    attr.mq_curmsgs = 0;

    if((mqd_sender = mq_open(mq_name, O_WRONLY | O_CREAT,
MQ_PERMISSIONS, &attr)) == (mqd_t)-1)
    {
        perror("Sender: mq_open");
        exit(1);
    }
    else
    {
        printf("Sender MQ %s opened\n", mq_name);
    }

    for(int i=1;i<=9;i++)
```